gemstone-py User Manual

Core APIs, transactions, bulk helpers, inspection, migrations, and web integration

Generated from the Markdown sources in gemstone-py/docs
Assets are local SVG illustrations stored in the repository.

Contents

User Manual

This manual explains the public surface of `gemstone-py` from the point of view of a user who wants to build reliable code, not just make a demo print a nice dictionary once.

The Mental Model

Three ideas matter more than the rest:

1. Python code drives the application.
2. GemStone stores the persistent state.
3. Transactions are explicit enough that you can explain them to another human.

`gemstone-py` is not trying to hide GemStone completely. It gives you a Pythonic surface, but it still wants you to understand:

- when sessions are opened and closed
- when transactions commit or abort
- which data structures are persisted in GemStone
- what happens when two sessions try to change the same thing

That restraint is a strength. The library does not pretend that distributed state is a teddy bear.

Core Building Blocks

`GemStoneConfig`

`GemStoneConfig` is the runtime configuration object. In most applications you will construct it from the environment:

```
from gemstone_py import GemStoneConfig

config = GemStoneConfig.from_env()
```

Use it when:

- you want a single source of connection settings
- you need to pass the same config into sessions, stores, or web providers
- you want to validate missing credentials early

`GemStoneSession`

This is the direct session object. Use it when you want explicit session and transaction control.

Typical pattern:

```
from gemstone_py import GemStoneSession, TransactionPolicy

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
) as session:
    print(session.eval("System myUserProfile userId"))
```

Useful methods include:

- `eval(...)`

  Evaluate Smalltalk code directly.

- `commit()`

  Commit the current transaction.

- `abort()`

  Abort the current transaction.

- `global_get(...)`

  Resolve named objects from the repository.

- `bulk_perform_value(...)` / `bulk_perform_oop(...)`

  Send one selector to many raw OOP receivers in one evaluated batch.

- `bulk_perform_calls_value(...)` / `bulk_perform_calls_oop(...)`

  Send mixed selector calls in one batch when each receiver or selector differs.

When to use it:

- scripts
- lower-level tooling
- integration code that must control failure behaviour precisely

Bulk Selector Sends

Bulk sends are for the cases where you already have raw OOPs and would otherwise call `perform_*` repeatedly in a loop. They keep the API explicit: you still choose selectors and raw OOP arguments, but the package evaluates the batch in one GemStone round trip.

```
from gemstone_py import PerformCall

with GemStoneSession(config=config) as session:
    order_oops = [session.global_get("OrderA"), session.global_get("OrderB")]
    statuses = session.bulk_perform_value(order_oops, "status")

    labels = session.bulk_perform_calls_value([
        PerformCall(order_oops[0], "printString"),
        (order_oops[1], "status"),
    ])
```

Use `perform_many_value(...)` and `perform_many_oop(...)` when that reads more naturally in application code. They are aliases for the bulk selector helpers. `AsyncSession` exposes the same bulk and perform-many names as `async` methods.

`session_scope(...)`

This is the higher-level unit-of-work helper. It is usually the right answer for application code.

```
from gemstone_py import session_scope

with session_scope(config=config) as session:
    ...
```

Why it exists:

- it makes commit-on-success intent obvious
- it composes naturally with service-layer code
- it avoids sprinkling `commit()` everywhere like nervous confetti

`AsyncSession`

`gemstone_py.aio.AsyncSession` is the async facade for applications built on `asyncio`, FastAPI, or Starlette-style request handling.

It does not make GCI itself async. Instead, it runs all calls for one session on one dedicated worker thread. That keeps the GemStone session on its owning thread while letting your event loop continue serving other work.

```
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession, AsyncSessionPool
```

```
config = GemStoneConfig.from_env()

async with AsyncSession.connect(config=config) as session:
    value = await session.eval("3 + 4")

    async with session.transaction():
        root = session.root()
        await root.set("AsyncManualExample", {"value": value})
```

Use `AsyncSession` when:

- request handlers are already async
- you want FastAPI dependency injection
- you need to keep blocking GCI work out of the event loop

Use `AsyncSessionPool` when async requests should reuse logged-in GemStone sessions instead of opening one session per request:

```
pool = AsyncSessionPool(
    maxsize=8,
    minsize=2,
    config=config,
    idle_timeout_seconds=900,
    validation_query="1 + 1",
    validation_interval_seconds=60,
)

async with pool.acquire() as session:
    value = await session.eval("3 + 4")

await pool.close()
```

The runnable repository example is `examples/async_features/session_root_and_collection.py`.

Transaction Policies

`TransactionPolicy` exists so you do not have to guess what a context manager will do.

The main values are:

- `MANUAL`
- `COMMIT_ON_SUCCESS`
- `ABORT_ON_EXIT`

Recommended use:

Situation	Policy
low-level explicit control	`MANUAL`
script that writes data	`COMMIT_ON_SUCCESS`
read-only inspection	`ABORT_ON_EXIT`
request/response web unit of work	`COMMIT_ON_SUCCESS` via `session_scope(...)` or request integration

`PersistentRoot`

`PersistentRoot` is the friendliest entry point into repository-backed state.

The default instance is the user's `UserGlobals` dictionary:

```
from gemstone_py.persistent_root import PersistentRoot

root = PersistentRoot(session)
root["CustomerCount"] = 12
```

Other dictionaries are also available:

- `PersistentRoot.globals(session)`
- `PersistentRoot.published(session)`
- `PersistentRoot.session_methods(session)`

Use `PersistentRoot` when:

- you want a stable named entry point
- you are storing dictionaries, counters, queues, or domain objects
- you want something simple and direct

Use something more specialized when:

- you need indexed search across many rows -> `GSCollection`
- you want a file-like key/value store abstraction -> `GStore`
- you want append-only event style logging -> `ObjectLog`

Good Practices With `PersistentRoot`

- keep the top-level key space tidy
- use descriptive names
- group related data under one top-level bucket when it makes sense
- do not turn `UserGlobals` into a junk drawer with no naming discipline

Batch Reads And Writes

Use `get_many(...)` and `update_many(...)` when a request, migration, or maintenance script needs several keys at once. The helpers avoid a Python loop of individual repository sends while keeping the operation visible in code.

```
with session_scope(config=config) as session:
    root = PersistentRoot(session)

    values = root.get_many(["CustomerCount", "Settings"], default=None)
    root.update_many({
        "CustomerCount": int(values["CustomerCount"] or 0) + 1,
        "LastSweepAt": "2026-05-14T22:00:00Z",
    })

    if "Settings" not in root:
        root["Settings"] = {}
    settings = root["Settings"]
    settings.update_many({"theme": "dark", "page_size": 50})
```

`PersistentRoot` uses symbol keys for top-level `UserGlobals` access. Nested `GsDict` values use string keys. That distinction mirrors the GemStone objects being wrapped rather than hiding them behind object mapping. `AsyncPersistentRoot` exposes async `get_many(...)` and `update_many(...)` counterparts for async applications.

For persistent ordered lists, `OrderedCollection.extend(...)` adds several values with one `addAll:` send:

```
from gemstone_py import session_scope
from gemstone_py.ordered_collection import OrderedCollection

with session_scope(config=config) as session:
    collection = OrderedCollection(session)
    collection.extend(["draft", "review", "paid"])
    root = PersistentRoot(session)
    root["InvoiceStates"] = collection
```

GSCollection

`GSCollection` is the indexed collection helper. It is the right tool when you need more than "look up a dictionary by a single well-known key."

Use it for:

- indexed search
- filtering by stored attributes
- application collections with repeatable lookup patterns

Typical pattern:

```
from gemstone_py.gsquery import GSCollection

people = GSCollection("People", config=config)
people.insert({"@name": "Tariq", "@city": "London"})
people.add_index("@city")
matches = people.search("@city", "eql", "London")
```

Think of it as the package's middle ground between:

- raw repository objects
- and a full ORM that wants to restructure your life

Typed Queries

`GSCollection.query(...)` can accept a Python `Protocol` as a type witness. The query still runs against GemStone indexed paths, but your Python code gets attribute names and better editor help.

```
from typing import Protocol
from gemstone_py.gsquery import GSCollection

class BlogPostRecord(Protocol):
    title: str
    status: str

posts = GSCollection("SimplePosts", config=config).query(BlogPostRecord)
published = posts.where(lambda post: post.status == "published").all()

for post in published:
    print(post.title)
```

For large result sets, use `iter(chunk_size=...)` to fetch records in bounded batches. `all()` keeps the familiar list return value, but it materializes that list by consuming the same chunked iterator path:

```
for post in posts.where(lambda post: post.status ==
"published").iter(chunk_size=500):
    print(post.title)
```

The lambda is a query builder expression, not a live object callback. `post.status` becomes the GemStone ivar path `@status`.

See `examples/typed_access/typed_oops_and_queries.py` for a runnable typed query and `examples/typed_access/simple_blog_queries.py` for importable helper functions.

Typed OOPs and Lifetime Handles

Raw integer OOPs remain supported for compatibility. New code can opt into typed or managed wrappers when the extra structure is useful.

`TypedOop[T]` carries a phantom Python type:

```
from typing import Protocol
from gemstone_py import GemStoneSession, gemstone_class

@gemstone_class("Date")
class GemStoneDate(Protocol):
    @property
    def printString(self) -> str: ...

with GemStoneSession(config=config) as session:
    today = session.execute_typed("Date today", GemStoneDate)
    print(today.proxy().printString)
```

Generated Typed Wrappers

When a Protocol describes method-shaped GemStone access, use `gemstone-codegen` to generate a concrete wrapper instead of repeating Smalltalk strings throughout the application.

```
from typing import Protocol
from gemstone_py import gemstone_class, gemstone_selector

@gemstone_class("OkzBooking", async_=True)
class OkzBookingProto(Protocol):
    status: str

    @classmethod
    @gemstone_selector("findById:")
    def find_by_id(cls, booking_id: str) -> "OkzBookingProto":
        ...

    def mark_paid(self, at_posix_seconds: int) -> None:
        ...
```

Generate and check in the wrapper package:

```
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --clean
```

Application code can then use regular Python methods:

```
from examples.typed_access.codegen_demo.generated import OkzBooking

with GemStoneSession(config=config) as session:
    booking = OkzBooking.find_by_id(session, "B-1001")
    print(booking.status)
    booking.mark_paid(1_779_912_000)
```

See `docs/codegen.md` for selector rules, async wrappers, CI `--check` usage, and the VS Code Codegen Explorer flow that consumes the database explorer's `/codegen/*` endpoints for live discovery, source lookup, selection JSON, and preview generation. Generated classes also expose `__gemstone_protocol__` and `__gemstone_selectors__` so diagnostics, admin UI, and code review tooling can identify where a wrapper came from without parsing generated source files.

For object lifetime, use `execute_managed(...)` when a raw OOP should stay in GemStone's export set while Python holds a handle:

```
with GemStoneSession(config=config) as session:
    collection = session.execute_managed("OrderedCollection new")
    collection.send("add:", session.new_string("kept alive"))
    print(collection.print_string())
    collection.close()
```

Use `session.handle(oop)` when you already have a raw OOP and want an explicit scope:

```
with session.handle(raw_oop) as handle:
    print(handle.send("printString"))
```

`execute()` and `perform()` still return raw OOP integers. The managed variants are additive and make lifetime intent visible.

The runnable repository example is `examples/lifetime/managed_oop_handles.py`.

Explicit Value Converters

`gemstone-py` does not globally convert every Python object it sees. When a GemStone API expects scalar-ish OOP arguments, opt into a small converter registry at the call site:

```
from datetime import date
from decimal import Decimal
from gemstone_py import scalar_value_converter_registry

registry = scalar_value_converter_registry()

with GemStoneSession(config=config) as session:
    invoice_oop = session.global_get("CurrentInvoice")
    due_on, amount = registry.to_oops(session, [
```

```
        date(2026, 5, 15),
        Decimal("19.95"),
    ])
    session.perform_oop(invoice_oop, "dueOn:amount:", due_on, amount)
```

The built-in registry covers `datetime`, exact `date`, `Decimal`, and `UUID`. Use `registry.copy()` and `registry.extend(...)` when an application needs its own explicit adapters.

For dataclasses, convert to a plain payload before persisting or sending it:

```
from gemstone_py import dataclass_to_dict

payload = dataclass_to_dict(booking_patch, recurse=False)
root["PendingBookingPatch"] = payload
```

This is deliberately value conversion, not object mapping. It does not create an identity map, track dirty fields, or decide when domain objects should be written.

GStore

`GStore` is a GemStone-backed key/value store with its own transactional behaviour around store operations.

Use it when:

- you want a store-like API
- you want separate named stores
- you want a compact way to persist structured payloads

Typical pattern:

```
from gemstone_py.gstore import GStore

store = GStore("inventory.db", config=config)
store["sku:123"] = {"name": "Hat", "stock": 8}
print(store["sku:123"])
```

`GStore` is especially handy in examples, utilities, and small application components that want a store-shaped abstraction without building a full domain model first.

`ObjectLog`

`ObjectLog` is the package's event-log helper. It is good for:

- audit trails
- structured append-only logging
- "what happened?" questions after a workflow ran

Typical pattern:

```
from gemstone_py.objectlog import ObjectLog

log = ObjectLog(config=config)
log.info("user_created", {"user": "tariq", "source": "manual"})
for entry in log.entries():
    print(entry)
```

This is not a replacement for Python logging. It is a repository-resident event log for when the record should live with the data.

Concurrency Helpers

The concurrency helpers give you shared, repository-backed coordination primitives:

- `RCCounter`
- `RCHash`
- `RCQueue`

These are useful when you need shared mutable state between sessions and want the concurrency semantics to live in GemStone instead of in a Python process.

Use them carefully:

- they are powerful
- they make contention visible
- they will teach you honesty about multi-session behaviour

The package also includes:

- nested transaction helpers
- conflict detection helpers
- instance listing utilities

Inspect And Debug

Use inspection helpers when you have a raw OOP or class name and need to see what GemStone is actually holding:

```
with GemStoneSession(config=config) as session:
    booking = session.execute("OkzBooking findById: 'B-1001'")

    inspected = session.inspect(booking)
    print(inspected.class_name, inspected.summary)

    dump = session.dump(booking, depth=2)
    print(dump["slots"])

    description = session.describe_class("OkzBooking")
    print(description.instvars)
```

The command-line form uses the same implementation:

```
gemstone-inspect --oop 123456789
gemstone-inspect --oop 123456789 --dump --depth 2
gemstone-inspect --class OkzBooking --json
```

`inspect()` returns a one-level `InspectionResult`; slot values are `InspectedReference` objects with OOP, class name, and summary. `dump()` follows those references recursively and marks cycles. Use `slots=...`, `classes=...`, and `max_slots=...` when the object graph is large or when a CLI report should stay readable:

```
dump = session.dump(
    booking,
    depth=3,
    slots=["status", "customer"],
    classes=["OkzBooking", "OkzCustomer"],
    max_slots=20,
)
```

The equivalent CLI flags are `--slot`, `--include-class`, `--max-slots`, and `--json`.

Migrations

Use module-style migrations when repository objects or GemStone-side classes need to evolve in a controlled order. A manifest lists migration modules, and each module exposes `upgrade(session)` plus optional `downgrade(session)`.

```
gemstone-migrations scaffold add_amount_to_booking --directory migrations
gemstone-migrations status --manifest my_app.migrations.manifest
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run --record
gemstone-migrations upgrade --manifest my_app.migrations.manifest
```

Plain `--dry-run` reports the pending migration ids. `--dry-run --record` also runs migration callbacks against a `RecordingMigrationSession`, prints common session calls, and never sends those calls to GemStone:

```
dry-run upgrade: 1 step(s)
  002_add_amount_to_booking
recorded operations:
  # upgrade 002_add_amount_to_booking
  session.eval("OkzBooking addInstVarName: 'amount' ifAbsent: [ nil ]")
  session.commit()
```

The recorder is deliberately conservative. It returns placeholder OOPs and is best for migrations made of direct `session.eval(...)`, `execute(...)`, `perform_value(...)`, `commit()`, and `abort()` calls. Use a staging stone for migrations that branch on live data.

Applied versions are tracked under `GemstonePyMigrations` in `UserGlobals`. Before applying new steps, the runner checks that the live version table still matches the local manifest and stored migration checksums.

Real `upgrade` and `downgrade` runs acquire an advisory lock in `UserGlobals` before applying steps. Dry-runs do not lock. If a previous process crashed and left a stale lock, use `--force-lock` with a clear owner string:

```
gemstone-migrations upgrade --manifest my_app.migrations.manifest \
  --force-lock --lock-owner release-2026-05-11
```

For class-shape changes, compare a live GemStone class with a local Protocol or type witness:

```
gemstone-migrations diff-class OkzBooking \
  --local-class my_app.models:OkzBookingProto
```

The output is advisory: review the suggested instance-variable changes before copying them into a migration.

For deployment checks, fingerprint the expected repository shape without mutating the stone:

```
gemstone-migrations fingerprint \
  --root Bookings \
  --root BookingIndexes \
  --class OkzBooking \
```

```
  --manifest my_app.migrations.manifest \
  --json
```

Application startup or release automation can use the Python API when it needs to compare against a known digest:

```
from gemstone_py.migrations import assert_schema_fingerprint, load_manifest

steps = load_manifest("my_app.migrations.manifest")
assert_schema_fingerprint(
    session,
    expected_digest="...",
    root_keys=["Bookings", "BookingIndexes"],
    class_names=["OkzBooking"],
    migration_steps=steps,
)
```

The live test lane includes a module migration round trip:

```
GS_RUN_LIVE=1 ./scripts/run_live_checks.sh
```

Commit Conflicts

When two sessions modify the same object and both try to commit, GemStone raises a conflict. The second committer gets a `CommitConflictError`.

This is not a bug. It is the correct behaviour of an optimistic concurrency system. The right response is:

1. catch `CommitConflictError`
2. call `session.abort()` to reset the transaction
3. reload the data
4. reapply the change (with a narrowed scope if possible)
5. retry the commit

Keep retries bounded. Log them. If you see frequent conflicts on the same object, that is a signal the write pattern needs rethinking — not that the concurrency model is wrong.

```
from gemstone_py import GemStoneConfig, PersistentRoot, retrying_transaction

config = GemStoneConfig.from_env()

def increment_visits(session):
    root = PersistentRoot(session)
    root["Visits"] = int(root.get("Visits", 0)) + 1
    return root["Visits"]
```

```
new_value = retrying_transaction(increment_visits, config=config, attempts=5)
```

The retry helper takes a callable rather than a context manager because a real retry must replay the whole unit of work after aborting and reloading state. Use `on_conflict=` when you want structured logging:

```
def log_conflict(retry):
    print(retry.format(include_summaries=False))

retrying_transaction(
    increment_visits,
    config=config,
    attempts=5,
    on_conflict=log_conflict,
)
```

For lower-level handlers, `CommitConflictError` exposes `diagnostics(session)` and `format(session)`. The top-level `format_commit_conflict(...)` helper renders the same report when you already have the exception object.

Web Integration

The web integration lives in `gemstone_py.web`.

The main pieces are:

- `install_flask_request_session(...)`
- `GemStoneSessionPool`
- `GemStoneThreadLocalSessionProvider`
- `session_scope(...)`
- `gemstone_py.session_providers`
- `gemstone_py.frameworks.django`
- `gemstone_py.frameworks.flask`
- `gemstone_py.web_core.RequestScope`
- `gemstone_py.web_core.AsyncRequestScope`

The Flask and FastAPI adapters now share `gemstone_py.web_core`, a framework-neutral lifecycle layer. `RequestScope` and `AsyncRequestScope` own the lazy session acquisition, commit-or-abort decision, and provider release or logout path. Use those primitives when adding another framework adapter instead of copying Flask teardown code. The reusable sync providers live in `gemstone_py.session_providers`; the older `gemstone_py.web` and top-level imports remain compatibility re-exports.

One Session Per Request

If you are building a Flask app, use the request integration so the request lifecycle decides the final transaction outcome.

That avoids the worst class of web persistence bugs:

- request handled an exception
- response still rendered
- persistence layer silently committed partial work anyway

The package already corrected this behaviour at the framework layer, so use it.

Pool vs Thread-Local

Use `GemStoneSessionPool` when:

- you have multiple request workers
- you want bounded reuse
- you want production-friendly pooling semantics
- stale sessions should be checked with a cheap validation query before reuse
- idle sessions should be evicted instead of sitting in the pool indefinitely

```
provider = GemStoneSessionPool(
    maxsize=4,
    minsize=1,
    config=GemStoneConfig.from_env(),
    idle_timeout_seconds=900,
    idle_sweep_interval_seconds=60,
    validation_query="1 + 1",
    validation_interval_seconds=60,
)

stats = provider.stats()
print(stats.in_use, stats.idle, stats.created_total, stats.evicted_total)
```

`minsize` is the idle-retention floor for maintenance: sweeps will not evict below that many live sessions. Use `warm(count)` or Flask's `warmup_sessions` when you want eager login at application startup.

Use `GemStoneThreadLocalSessionProvider` when:

- your threading model is simple and stable
- one session per thread is the clearest fit

FastAPI

FastAPI integration lives in `gemstone_py.aio.fastapi`.

```
from fastapi import Depends, FastAPI
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession, AsyncSessionPool
from gemstone_py.aio.fastapi import pool_session_dependency, session_dependency

app = FastAPI()
get_gemstone = session_dependency(config=GemStoneConfig.from_env())

@app.get("/health/gemstone")
async def gemstone_health(session: AsyncSession = Depends(get_gemstone)):
    return {"result": await session.eval("3 + 4")}
```

For busier FastAPI apps, use a pool-backed dependency:

```
pool = AsyncSessionPool(maxsize=8, minsize=2, config=GemStoneConfig.from_env())
get_pooled_gemstone = pool_session_dependency(pool)
```

Use this when your web stack is async-first. Use the Flask integration when your application is Flask-first.

The runnable repository example is `examples/fastapi/app.py`.

Django

Django integration lives in `gemstone_py.frameworks.django`. The module does not import Django directly; it exposes middleware and a `request_session(...)` helper that work with Django's standard request object.

```
from django.http import JsonResponse
from gemstone_py import GemStoneConfig
from gemstone_py.frameworks.django import GemStoneSessionMiddleware, request_session

def gemstone_session_middleware(get_response):
    return GemStoneSessionMiddleware(get_response, config=GemStoneConfig.from_env())

def health(request):
    session = request_session(request)
    return JsonResponse({"result": session.eval("3 + 4")})
```

Install the optional framework dependency with:

```
python -m pip install "gemstone-py[django]"
```

Litestar

Litestar integration lives in `gemstone_py.aio.litestar`. The module does not import Litestar directly; it provides async dependency callables and ASGI middleware that application code can wire into Litestar.

```
from litestar import Litestar, get
from litestar.di import Provide
from gemstone_py import GemStoneConfig
from gemstone_py.aio.litestar import pool_session_dependency, session_dependency

get_gemstone = session_dependency(config=GemStoneConfig.from_env())
get_pooled_gemstone = pool_session_dependency(pool)

@get("/health/gemstone", dependencies={"session": Provide(get_gemstone)})
async def gemstone_health(session):
    return {"result": await session.eval("3 + 4")}

app = Litestar(route_handlers=[gemstone_health])
```

Install the optional framework dependency with:

```
python -m pip install "gemstone-py[litestar]"
```

The runnable repository example is `examples/litestar/app.py`. Use `gemstone-examples list` or `examples/cookbook/README.md` when choosing between FastAPI, Litestar, Flask, Django, and lower-level examples.

Adapter Core

For a custom sync framework, keep one `RequestScope` in request-local state and finalize it during teardown:

```
from gemstone_py import GemStoneConfig, RequestScope, TransactionPolicy
from gemstone_py.session_providers import GemStoneSessionPool

pool = GemStoneSessionPool(maxsize=4, config=GemStoneConfig.from_env())

scope = RequestScope(
    session_provider=pool,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
)
session = scope.session()
try:
    session.eval("3 + 4")
finally:
    scope.finalize()
```

For an async framework, use `AsyncRequestScope` with `AsyncSessionPool` and await `scope.finalize(...)` from the framework's request cleanup hook.

See `docs/framework-adapters.md` for a concise sync and async adapter recipe.

Benchmarks and Build Lanes

This package has both examples and maintained operational lanes. Keep them separate in your head.

Examples are for:

- learning
- translation exercises
- inspection

The benchmark lane is for:

- reproducible measurement
- stored JSON reports
- baseline comparison
- GitHub workflow enforcement

The CLI is:

```
gemstone-benchmarks --entries 500 --search-runs 20
```

Release and Packaging Surface

The public install surface is the `gemstone_py` package.

The package now has:

- PyPI publishing
- TestPyPI rehearsal
- post-release verification against real PyPI
- installed-artifact API contract checks
- local PyPI/TestPyPI index verification with `gemstone-publish-verify`
- optional PyO3 native wheels through `gemstone-py[fast]`

That means you can treat the package as a real distributable unit, not just a working directory with ambition.

After publishing, verify both public indexes from a clean shell:

```
gemstone-publish-verify --gemstone-version 0.2.12 --native-version 0.1.2
```

The command checks project JSON, version-specific JSON, the simple index, and temporary-virtualenv installs for PyPI and TestPyPI.

Native Fast Path

The default install uses the pure-ctypes GCI backend:

```
python -m pip install gemstone-py
```

The optional native fast path installs the PyO3 extension package:

```
python -m pip install "gemstone-py[fast]"
```

Backend selection happens at import time. For comparisons, set `GEMSTONE_PY_GCI_BACKEND=ctypes` or `GEMSTONE_PY_GCI_BACKEND=native` before starting Python.

The runnable repository example is `examples/native_backend/check_backend.py`.

For the Rust-backed shared-core bridge inside `gemstone-py-native`, run:

```
python scripts/run_native_rust_core_live_smoke.py --dry-run
GS_RUN_LIVE=1 python scripts/run_native_rust_core_live_smoke.py --require-live
```

Use the dry run in wheel/sdist packaging checks. Use the live command before publishing native wheels that are expected to exercise `gemstone_rs::py_native`.

Recommended Adoption Path

If you are bringing `gemstone-py` into a project, a sensible order is:

1. start with `GemStoneConfig` + `GemStoneSession`
2. move most application work into `session_scope(...)`
3. use `PersistentRoot` first for obvious top-level state
4. introduce `GSCollection` only when indexed queries are justified
5. add `TypedOop[T]`, typed queries, or managed handles when the code benefits

   from clearer object identity and lifetime
6. use Flask or FastAPI providers once request lifecycles matter
7. install `gemstone-py[fast]` only after the ctypes path is working

8. add benchmarks and live tests once the system is real

That order keeps the learning curve honest without making the first week feel like a database theology seminar.

Manual Summary

If you remember only five things:

1. Be explicit about transaction policy.
2. Use `PersistentRoot` first unless you need something more specialized.
3. Use async wrappers for async frameworks, not for shared-session threading.
4. Use typed and managed OOP wrappers when raw integers hide too much intent.
5. The examples teach the package; the maintained workflows prove it.

> This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.